

Name : Bhavya rana

Sap id 590034689

Batch : 15

1. Algorithm

Algorithm: Menu-Driven Arithmetic Tables

1. Start.
2. Declare integer variables `m`, `n`, `choice`, `i`, and `j`.
3. Input two integers `m` and `n`.
4. Check whether:
 - `m > 0`
 - `n > 0`
 - `m <= n`
5. If the input is invalid, display an error message and repeat Step 3 until valid values are entered.
6. Display the menu:
 - 1. Multiplication tables
 - 2. Addition tables
 - 3. Subtraction tables
 - 4. Division tables
 - 5. Exit
7. Input `choice`.
8. Use a `switch` statement:
 - **Case 1:** For every integer `i` from `m` to `n`, use an inner loop from `j = 1` to `10` and display `i * j`.
 - **Case 2:** For every integer `i` from `m` to `n`, use an inner loop from `j = 1` to `10` and display `i + j`.
 - **Case 3:** For every integer `i` from `m` to `n`, use an inner loop from `j = 1` to `10` and display `i - j`.
 - **Case 4:** For every integer `i` from `m` to `n`, use an inner loop from `j = 1` to `10` and display `i / j` rounded to two decimal places.
 - **Case 5:** Display an exit message.
 - **Default:** Display an invalid menu-choice error message.
9. Repeat Steps 6–8 until the user selects 5.
10. Stop.

2. Test-Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1 – Valid Multiplication	m=2, n=2, choice=1	Multiples of 2 from $2 \times 1 = 2$ to $2 \times 10 = 20$	Multiples printed correctly	Pass
2 – Valid Addition	m=2, n=2, choice=2	Addition table from $2 + 1 = 3$ to $2 + 10 = 12$	Addition table printed correctly	Pass
3 – Valid Subtraction	m=2, n=2, choice=3	Subtraction table from $2 - 1 = 1$ to $2 - 10 = -8$	Subtraction table printed correctly	Pass
4 – Valid Division	m=2, n=2, choice=4	Division table from $2 / 1 = 2.00$ to $2 / 10 = 0.20$	Division table printed correctly	Pass
5 – Range Input	m=2, n=3, choice=1	First 10 multiples of both 2 and 3	Both tables printed correctly	Pass
6 – Boundary m=n	m=1, n=1, choice=1	First 10 multiples of 1	Correct table printed	Pass
7 – Invalid m	m=0, n=5	Error message and request input again	Error message	Pass
8 – Invalid n	m=2, n=-4	Error message and request input again	Error message	Pass
9 – Invalid m	m=5, n=2	Error because $m > n$; request input	Error message	Pass
10 – Invalid Choice	m=2, n=3, choice=9	Invalid menu choice. Please enter 1 to 5.	Correct error displayed	Pass
11 – Exit	m=2, n=3, choice=5	Exiting program.	Program exits correctly	Pass

3. C Program

```
#include <stdio.h>
```

```
int main(void)
{
```

```
    int m, n, choice, i, j;
```

```
    /* Input and validation of m and n */
```

```
    do
```

```
    {
```

```
        printf("Enter two positive integers m and n (m <= n): ");
```

```
        scanf("%d %d", &m, &n);
```

```
        if (m <= 0 || n <= 0 || m > n)
```

```
        {
```

```
            printf("Invalid input. Enter positive integers with m <= n.\n");
```

```
        }
```

```

} while (m <= 0 || n <= 0 || m > n);

/* Menu repeats until Exit is selected */
do
{
    printf("\nMENU\n");
    printf("1. Multiplication tables\n");
    printf("2. Addition tables\n");
    printf("3. Subtraction tables\n");
    printf("4. Division tables\n");
    printf("5. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            /* Multiplication tables */
            for (i = m; i <= n; i++)
            {
                printf("\nMultiples of %d:\n", i);

                for (j = 1; j <= 10; j++)
                {
                    printf("%d x %d = %d\n",
                        i, j, i * j);
                }
            }
            break;

        case 2:
            /* Addition tables */
            for (i = m; i <= n; i++)
            {
                printf("\nAddition table of %d:\n", i);

                for (j = 1; j <= 10; j++)
                {
                    printf("%d + %d = %d\n",
                        i, j, i + j);
                }
            }
            break;
    }
}

```

```

        case 3:
            /* Subtraction tables */
            for (i = m; i <= n; i++)
            {
                printf("\nSubtraction table of %d:\n",
i);

                for (j = 1; j <= 10; j++)
                {
                    printf("%d - %d = %d\n",
                        i, j, i - j);
                }
            }
            break;

        case 4:
            /* Division tables */
            for (i = m; i <= n; i++)
            {
                printf("\nDivision table of %d:\n", i);

                for (j = 1; j <= 10; j++)
                {
                    printf("%d / %d = %.2f\n",
                        i, j, (double)i / j);
                }
            }
            break;

        case 5:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid menu choice. "
                "Please enter 1 to 5.\n");
    }

    while (choice != 5);

    return 0;
}

```

5. Sample Execution and Testing

For $m = 2$, $n = 2$ and Choice 1:

Bhavya rana

590034689

Batch 15

```

[bhavya@Bhavyas-MacBook-Air ~ % clang day21.cpp -o day21
[bhavya@Bhavyas-MacBook-Air ~ % ./day21
Enter two positive integers m and n (m <= n): 2 2

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice: 1

Multiples of 2:
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit

```

For choice (4)

```

Enter your choice: 4

Division table of 2:
2 / 1 = 2.00
2 / 2 = 1.00
2 / 3 = 0.67
2 / 4 = 0.50
2 / 5 = 0.40
2 / 6 = 0.33
2 / 7 = 0.29
2 / 8 = 0.25
2 / 9 = 0.22
2 / 10 = 0.20

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit

```

For an invalid choice (9)

```
5. Exit
Enter your choice: 9
Invalid menu choice. Please enter 1 to 5.

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
```

For choice 5

```
Enter your choice: 5
Exiting program.
bhavya@Bhavyas-MacBook-Air ~ %
```

Final Test Result: All prepared test cases pass successfully.

